

Embedded Systems and Microcontrollers (CS1120)

Unit 1: Introduction to Embedded Systems and Memory

RV University — 2nd Semester CS | CIE 1 Master Notes

1. Embedded Systems Overview

1.1 What is a System?

- **Definition:** An arrangement in which all constituent units are assembled to operate together according to a fixed, pre-defined set of rules.
 - A system can also be defined as: a way of working, organizing, or performing one or more tasks according to a fixed plan.
 - **Example:** A digital watch — collects time input (oscillator), processes it, and displays the current time on an output device (display).
-

1.2 What is an Embedded System?

Formal Definition: An **embedded system** is a **microcontroller** or **microprocessor**-based computing system — normally battery-operated — that integrates both hardware and software, and is designed to perform a **specific, dedicated task**.

Key Properties:

- The word "*embedded*" literally means "attached to" or "integrated within" a larger entity.
- An **embedded system** can function:
 - As a **standalone** unit (e.g., a pacemaker), or
 - As a **sub-system** within a larger system (e.g., the engine control unit in a car).

- The system integrates both **hardware** (processor, memory, I/O interfaces) and **software** (**firmware**, application code, **RTOS**).

Real-World Examples:

Domain	Embedded System Examples
Consumer	Fire alarms, coffee dispensers, smart door locks, digital cameras
Automotive	Flight controllers, cruise control, ABS, airbag ECUs
Industrial	CNC machines, PLCs, industrial robots
Medical	Pacemakers, infusion pumps, blood glucose monitors
Communication	Smart phones, routers, modems
IoT	Smart home sensors, wearable devices

1.3 Embedded Software Architecture

The software architecture of an **embedded system** is structured as three concentric layers (innermost to outermost):

Application Software	← Outermost layer
Operating System	← Middleware (RTOS)
Hardware	← Innermost layer

- **Hardware Layer:** Processor core, memory (**Flash/SRAM**), peripherals (GPIO, ADC, UART, SPI, I2C), power management ICs.
- **Operating System Layer (RTOS):** A **Real-Time Operating System** — manages scheduling, resource allocation, inter-task communication, and timing guarantees.
- **Application Software Layer:** The domain-specific control/data processing logic written by the developer.

RTOS — Real-Time Operating System:

An OS specifically designed for **embedded systems** with **stringent constraints** on:

- **Execution time** — tasks must complete within hard deadlines
- **Memory usage** — RAM/**Flash** budgets are tight
- **Latency** — the delay between event and response is bounded
- **Performance** — deterministic behavior under all conditions

Key distinction: A general-purpose OS (like Linux/Windows) prioritizes *throughput*; an **RTOS** prioritizes *determinism* and *latency*.

1.4 Three Components of an Embedded System

Every **embedded system** typically contains exactly **three functional components**:

Component	Description
Hardware	Physical substrate — processor chip, PCB, sensors, actuators, power supply
Embedded Application Software	Domain logic — the firmware/program that implements the system's functionality
RTOS (or bare-metal runtime)	Supervises application execution; provides scheduling, synchronization, and I/O abstractions

Role of the RTOS in detail:

- Provides mechanisms (e.g., priority-based preemptive scheduling) to meet **latency requirements**
- Defines the rules governing how the application program executes at runtime
- Enforces time boundaries on task execution

Note: Small-scale **embedded systems** (e.g., a simple LED controller on an 8-bit AVR) may run without any **RTOS** — this is called **bare-metal** programming.

1.5 Bare Metal Systems

Bare Metal: A computer system in which **software or firmware runs directly on the hardware** with no operating system (OS) layer between them.

- No OS overhead → minimum latency, minimum memory footprint

- Used in resource-constrained, single-task systems (e.g., a washing machine controller, a simple sensor node)
 - The developer manages all hardware resources directly in code
 - **Contrasted with RTOS-based systems** where the OS abstracts resource management
-

2. Characteristics of Embedded Systems

Characteristic 1: Single-Functioned

- An **embedded system** is designed to execute **one specialized operation** and performs it **repeatedly** throughout its operational life.
- The functionality is fixed and does not change (unlike a general-purpose computer).
- **Example:** A pager *always* functions as a pager — it never acts as a spreadsheet processor.

Characteristic 2: Tightly Constrained

- All computing systems must satisfy **design metrics**, but **embedded systems** face **especially tight constraints** where missing a deadline can be catastrophic.

Design Metrics: A quantitative measure of an implementation's features across multiple dimensions.

Key design metric dimensions:

Metric	Embedded Constraint
Cost	Selling price must be low (commodity market); BOM cost is critical
Size	Must fit on a single chip or a small PCB
Power consumption	Battery-operated → must minimize power to maximize battery life
Latency / Response time	Real-time constraints — computation must finish within a deadline
Performance (throughput)	Must process data fast enough for the application (e.g., audio sampling)

Related acronyms (from slides):

Acronym	Full Form	Meaning
ASIC	Application-Specific Integrated Circuit	Custom chip designed for one specific application; not reprogrammable
ASSP	Application-Specific Standard Processor	Off-the-shelf chip designed for a specific application domain (e.g., a Wi-Fi SoC)
ASIP	Application-Specific Instruction Processor	Processor with a custom ISA tailored to a specific application

Characteristic 3: Reactive and Real-Time

- Many **embedded systems** must **continually react** to changes in the external environment.
- They must **compute certain results in real time** — without any delay — in response to those changes.

Example — Car Cruise Controller:

- Continuously monitors speed and brake sensors (reactive)
- Must compute acceleration/deceleration commands **repeatedly within a bounded time window** (real-time)
- A delayed computation → loss of vehicle control (catastrophic failure)

Characteristic 4: Microprocessor-Based

- The computational core must be a **microprocessor or microcontroller** on top of which application software (or **RTOS** + application) executes.
- This is what distinguishes an **embedded system** from simple analog/digital logic circuits.

Characteristic 5: Memory

- Embedded system software is typically stored in **Flash/ROM** (**non-volatile** program memory).
- Main memory is **DRAM or SRAM**:
 - **SRAM**: used for caches and internal CPU registers (fast, expensive)
 - **DRAM**: used for main working memory (slower, cheaper, larger)
- **No secondary storage (HDD)** in most **embedded systems** → reliability issues + higher power consumption.

- **Flash memory** is used instead of HDD when secondary storage is needed (e.g., SD cards in embedded Linux).
- Embedded systems typically **lack full virtual memory support** due to timing constraints — paging/swapping introduces unpredictable latency.
- Normally **no memory management unit (MMU)** operation in small **embedded systems**.

Characteristic 6: Connected

- Must be interfaced to **peripherals** for connectivity and I/O processing.
- **Example:** IoT devices must connect to a network (Wi-Fi, Bluetooth, Zigbee, LoRa).
- Common peripheral interfaces: GPIO, UART, SPI, I²C, USB, CAN, Ethernet, ADC/DAC.

Characteristic 7: HW-SW Co-Design (HW-SW Systems)

- Both hardware and software components work in tandem:
 - **Software** → used for high-end features, flexibility, configurability, and rapid iteration
 - **Hardware** → used for higher performance (dedicated accelerators), lower power, and security (hardware roots of trust)
- This co-design philosophy is fundamental to embedded engineering.

3. Features and Challenges of Embedded Systems

Features (Desirable Properties)

Feature	Description
Easily customizable	Adaptable to different environments and application requirements
Low power consumption	Designed for longer battery life; often uses sleep/low-power modes
Low cost	Selling price must be competitive in high-volume consumer markets
Enhanced performance	Must meet real-time constraints; deterministic execution
Highly reliable	Safety-critical systems (medical, automotive, aerospace) demand high reliability/uptime

Challenges

Challenge	Description
High development effort	Must be highly reliable → exhaustive testing, formal verification, certification (e.g., IEC 61508, ISO 26262)
Larger time to market	Long design-verify-test cycles slow product launches
Higher volume requirements	Unit economics only make sense at high production volumes
Low power operations	Balancing performance with power efficiency is a persistent engineering challenge

4. Hierarchical Levels of Computer Systems

4.1 Six-Level Abstraction Model

A computer system can be described across **six levels of abstraction** (from lowest/concrete to highest/abstract):

Level of Abstraction	Basic Components	Behavioural Description By
Circuit element	Transistors, etc.	Differential equations
Circuit	Gates, Flip-Flops	Boolean equations
Functional block	Registers, MUXes, ALUs, Micro-sequencer	Operations (register transfers, state sequencing)
Processor	Processor	Basic instructions
Computer system	Computer system	Instructions
OS	OS	OS commands

HW/SW Boundary:

Programming Language	← SW
Application Programming Interface	← SW
Operating System	← SW
Assembly Language	← SW/HW boundary
Computer Architecture	← HW
Digital Electronics	← HW

- **Above the assembly language level:** Software domain
- **Below (computer architecture + digital electronics):** Hardware domain
- The **Assembly Language** layer serves as the interface between HW and SW

4.2 Architecture Levels: Logic → System

From the diagram in the slides, levels of abstraction in computer architecture (increasing abstraction downward):

Level	Description	Example
Logic Level	Concrete architecture of functional blocks	Circuit diagram of an ALU (with gates, wires)
Functional Block Level	Abstract architecture of functional blocks	Block symbol "ALU" with input/output arrows
Architecture Level (Concrete)	Concrete architecture of processors	Processor block with register files, ALU, data paths
Architecture Level (Abstract)	Abstract architecture of processors	Single block "P" representing a processor
System Level (Concrete)	Concrete architecture of a computer system	Multiple processor blocks + memory + bus diagram
System Level (Abstract)	Abstract architecture of a computer system	Single block "C"

Course Focus (CS1120): This course operates at the **Architecture Level** and **System Level** — understanding how processors are organized and how they interact with memory and peripherals.

5. Computer and Processor Architecture

5.1 Computer Architecture: Definition and Subcategories

Definition: Computer architecture is a **set of rules and methods** that describe the **functionality, organization, and implementation** of computer systems.

The discipline has **three main subcategories**:

Subcategory 1: Instruction Set Architecture (ISA)

- Defines the **machine code** that a processor reads, decodes, and acts upon
- Specifies:
 - **Word size** (e.g., 32-bit, 64-bit)
 - **Endianness** (Big-Endian vs. Little-Endian)
 - **Memory addressing modes** (immediate, register, direct, indirect, indexed)
 - **Processor registers** (general-purpose, special-purpose, control)
 - **Data types** (integer, floating-point, SIMD vectors)
 - **Instruction formats** (opcode + operand fields)
- **Examples of ISAs:** x86 (Intel/AMD), ARM (Cortex-M, Cortex-A), MIPS, **RISC-V**

MIPS = Microprocessor without Interlocked Pipelined Stages

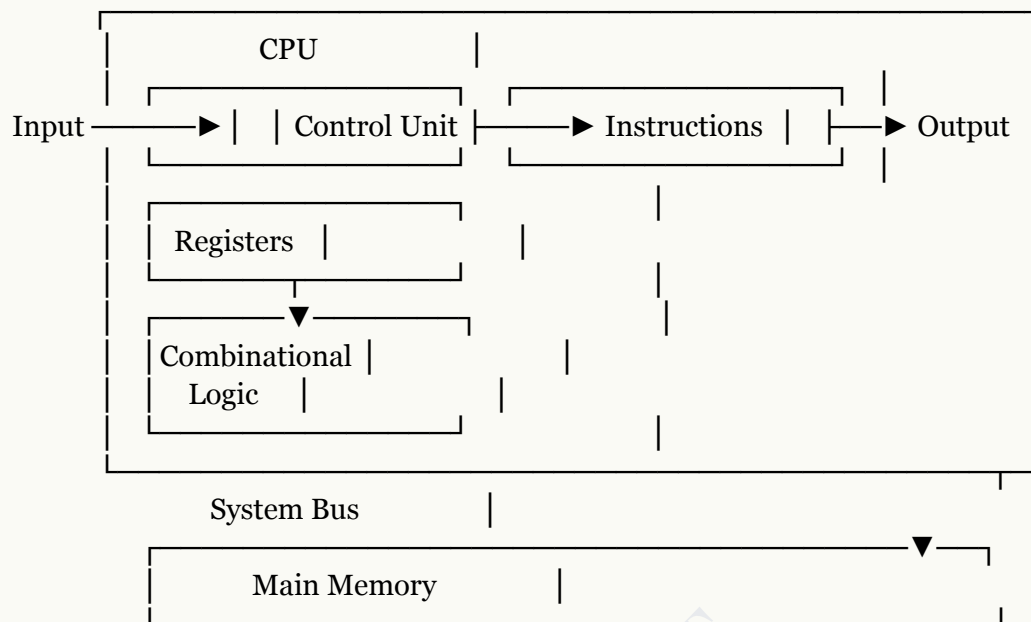
Subcategory 2: Microarchitecture (Computer Organization)

- Describes **how a specific processor implements the ISA** at the hardware level
- The same ISA can have multiple microarchitecture implementations (e.g., Intel Skylake, Ice Lake, Alder Lake all implement x86)
- Covers: pipelining, branch prediction, out-of-order execution, caching strategies
- **Important Note:** x86 processors (**CISC** architecture) internally **translate all CISC x86 instructions into RISC-type micro-operations (μ -ops)** before execution — this is a key microarchitectural technique

Subcategory 3: Systems Design

- Encompasses all hardware components **other than the CPU** within a computing system
- Includes: Cache hierarchy, Main Memory subsystem, Internal bus architecture, Peripheral controllers, I/O systems

CPU Block Diagram (from slides):



Black lines = data flows; Red lines = control flows; Arrows = direction of flow.

5.2 Processor Architecture: Definition and Subcategories

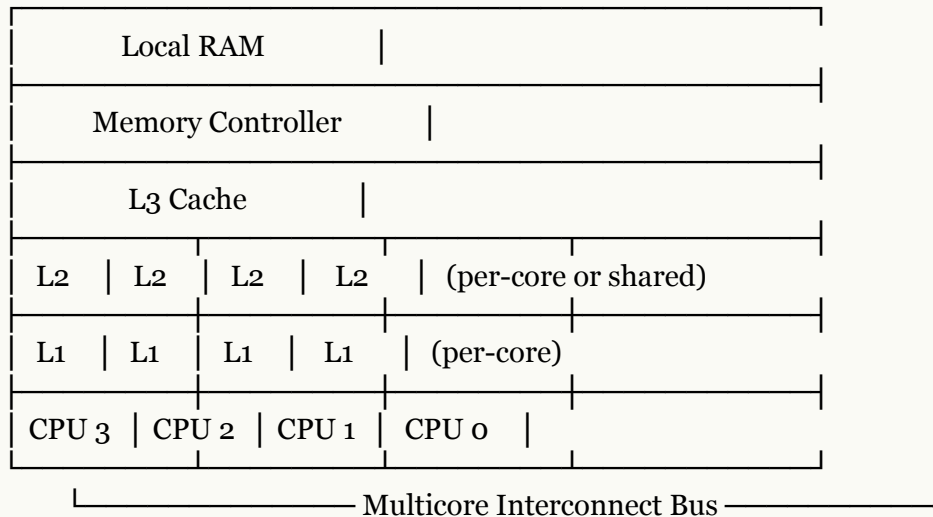
Definition: Processor Architecture details the **design of processors** (including multi-core), their **internal organization**, and their **interface with memory and peripherals**.

Processor architecture elaborates three aspects:

Aspect	Coverage
ISA	Instructions, word size, memory addressing modes, registers, data types
Microarchitecture	Implementation of the ISA (pipeline stages, execution units, decode logic)
Multicore Design Issues	Cache coherency between cores; internal buses interfacing with memory and peripherals

SoC (System on a Chip): A single integrated circuit that contains all functional blocks of a complete system — CPU cores, L1/L2/L3 caches, memory controller, GPU, I/O controllers, etc.

Multi-Core Processor (SoC) Block Diagram:



Note: Computer architecture is a *superset* of processor architecture — it also includes system-level components (memory, peripherals, buses).

6. Software Development Tools and Program Lifecycle

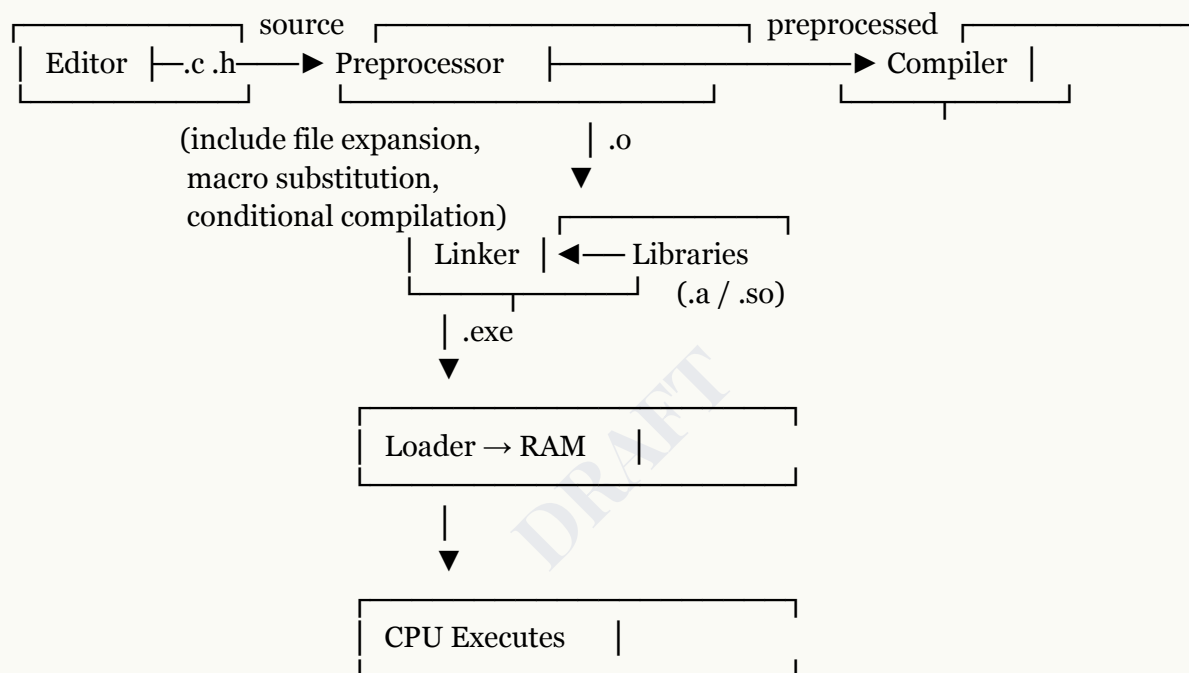
6.1 Phases of SW Program Development

A C program goes through **6 sequential phases** before it executes on the CPU:

Phase	Tool	Input	Output
1. Edit	Text editor / IDE	Developer keystrokes	.c and .h source files
2. Preprocessor	C Preprocessor (cpp)	.c + .h files	Expanded .c file (macros expanded, #include files inlined)
3. Compile	C Compiler (gcc/clang)	Preprocessed .c	.obj / .o object file (machine code, unlinked)

4. Link	Linker (ld)	<code>.o</code> files + library <code>.a/.so</code> files	Executable file (<code>.exe</code> / ELF binary)
5. Load	Loader (OS)	Executable file on disk	Program image in main memory
6. Execute	CPU	Instructions in memory	Program runs; CPU fetches, decodes, executes instructions

Expanded Description of Each Phase:



Preprocessor details:

- Handles `#include <stdio.h>` → pastes the contents of `stdio.h` into the source
- Expands `#define` macros
- Processes conditional compilation (`#ifdef`, `#ifndef`, `#endif`)

Compiler details:

- Translates C source → Assembly (`.asm`), then → machine code (`.o` binary object file)
- The `.o` file contains machine code but with **unresolved external symbol references** (e.g., calls to `printf`)

Linker details:

- Resolves symbol references by combining multiple `.o` files and library archives

- Produces a fully self-contained executable

Loader details:

- The OS **loader** reads the executable from disk and places the program's **code** and **static data** into main memory (**DRAM**) at the application load address
 - Then hands control to the program's entry point (`main()`)
-

6.2 From C Source to Executable: Step-by-Step

Example C snippet:

```
int count;  
count++;
```

Step 1 — Compile to Assembly (.asm):

```
ADR R1, count ; load the address of 'count' into register R1  
LDR R0, [R1] ; copy the content at address R1 (i.e., value of count) into R0  
ADD R0, R0, #1 ; increment R0 by 1  
STR R0, [R1] ; write the incremented value back to the address of count
```

Step 2 — Compile to Object File (.o):

1010011010 (binary machine code — unlinked object file)

Step 3 — Link to Executable (.exe):

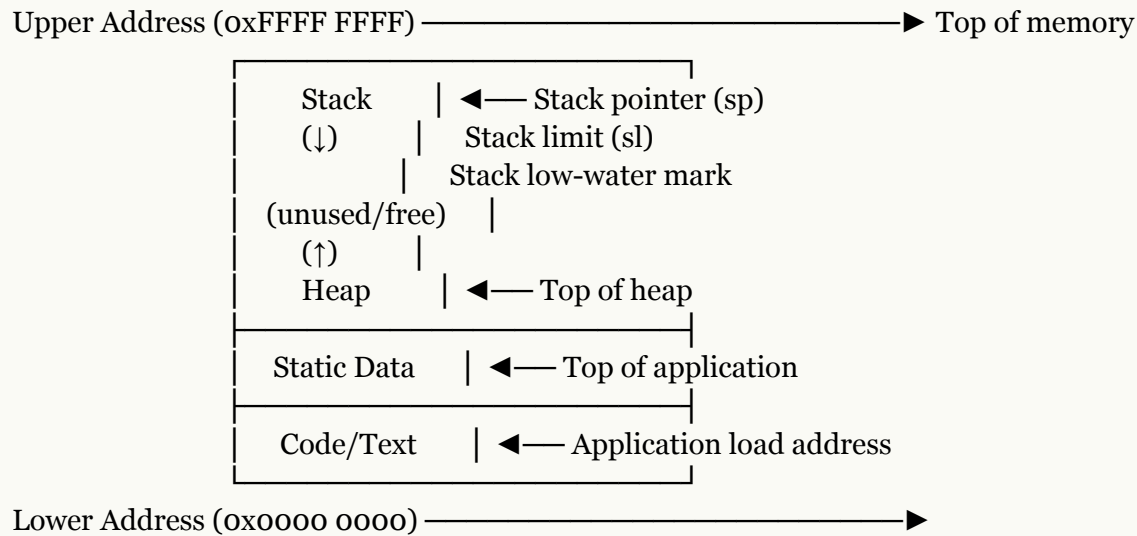
10111000011010 (fully linked binary — all external symbols resolved)

Step 4 — Load into memory, then Execute on CPU.

7. Memory Layout of a C Program

7.1 32-bit Processor Address Space Model

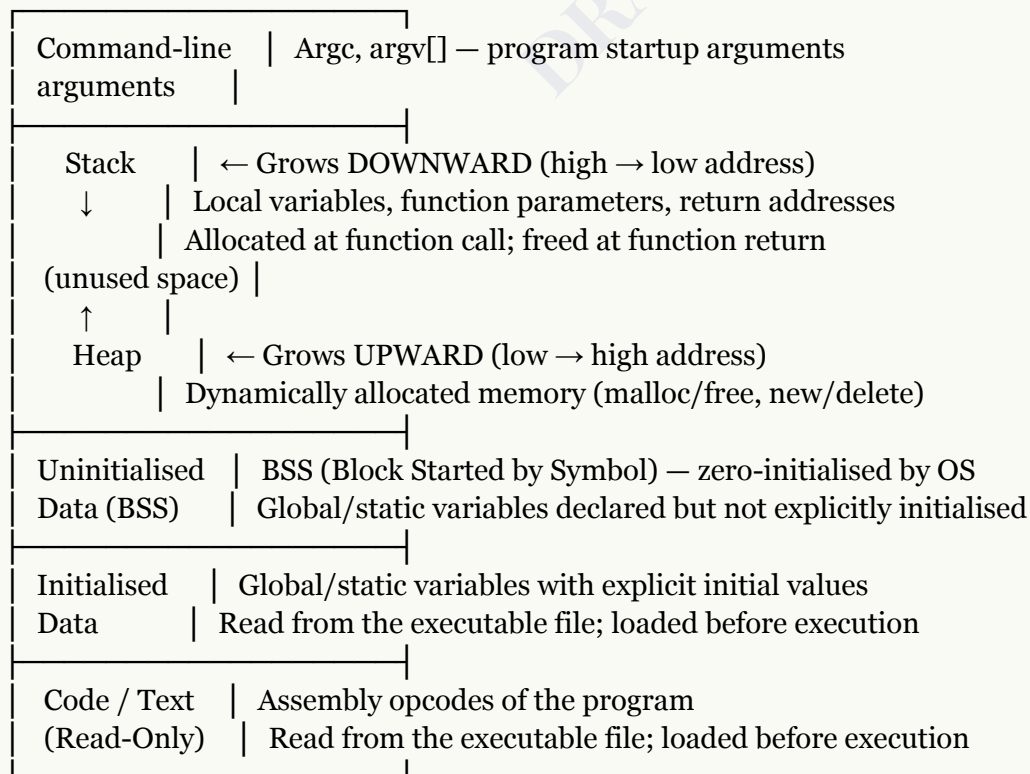
- A 32-bit processor can address $2^{32} = 4,294,967,296$ bytes = 4 GB of physical **address space**.
- Address range: `0x0000 0000` (Lower/bottom) → `0xFFFF FFFF` (Upper/top)



7.2 Typical Memory Segments

The OS prepares this memory layout in **DRAM** (main memory) by reading the executable from the hard disk, *before* handing control to the program. Here is a detailed breakdown of each segment:

Higher address



Annotated with the example program from slides:

```
#include <stdio.h>
#include <stdlib.h>

const float pi = 3.14; // → Initialised Data (ROM-constant, loaded from .exe)
static int s1 = 10;    // → Initialised Data
static int s2;         // → BSS (uninitialised static — zero-filled)
int g = 2;             // → Initialised Data

int inc(int i) {
    return i++;        // parameter 'i' → Stack (when inc() is called)
}

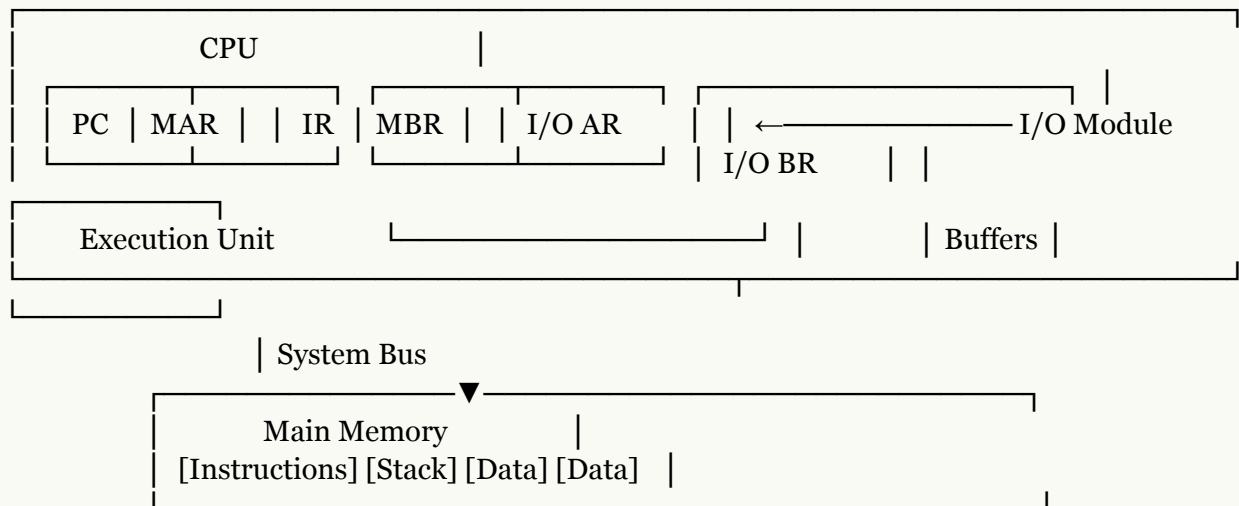
main(void) {
    int local = 10;     // → Stack (local variable of main())
    int *ip = NULL;     // → Stack (pointer itself is on stack)
    static int s3;      // → BSS (uninitialised static — zero-filled)
    ip = (int *) malloc(sizeof(int) * 10); // → Heap (10 int block allocated)
    local = inc(local);
    free(ip);
    return 0;
}
```

Variable	Location	Reason
pi, s1, g	Initialised Data	Global/static, explicitly initialised
s2, s3	BSS	Static, uninitialised → zero-filled
local, ip (pointer)	Stack	Local variables to main()
i (in inc)	Stack	Parameter pushed on call
malloc block (10 ints)	Heap	Dynamically allocated at runtime
All function opcodes	Code/Text	Compiled machine instructions

Key: Changes At Run-time → Stack and Heap grow/shrink dynamically during execution.
Key: Read From .exe File → Code/Text and Initialised Data are loaded once from disk.
Key: Loaded Before Execution → BSS is zeroed by the OS before the program starts.

8. Architectural View of a Computing System

A computer system at the architectural level contains **three functional modules** coordinated by a **control mechanism**:



8.1 Key CPU Registers

Register	Abbreviation	Function
Program Counter	PC	Holds the address of the next instruction to be fetched
Instruction Register	IR	Holds the currently executing instruction
Memory Address Register	MAR	Holds the address being read from or written to in memory
Memory Buffer Register	MBR	Holds the data being transferred to/from memory
I/O Address Register	I/O AR	Holds the address of the I/O device being accessed
I/O Buffer Register	I/O BR	Holds data being transferred to/from an I/O device

State Transitions of a Computing System: I/O ↔ Data Movement ↔ Data Processing ↔ Data Storage (HDD)

Critical observation from slides:

"The CPU needs to read each instruction and read/write data from/to the memory — is there an issue with it?" → Yes: The CPU is far faster than **DRAM**. This gives rise to the **CPU–DRAM Performance Gap** (Section 9).

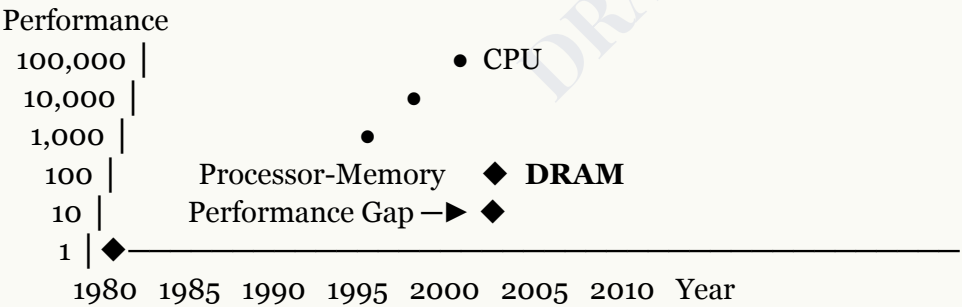
9. CPU–DRAM Performance Gap

The Problem:

Component	Annual Performance Improvement
CPU	~60% per year
DRAM	< 10% per year

This creates an **exponentially growing gap** between CPU speed and memory speed over time. The processor spends a large fraction of its cycles **waiting for data from DRAM** — a condition called a **memory stall** or **memory bottleneck**.

Trend (log scale):



Solution: Exploit the **Principle of Locality** (Section 10) to bridge this gap with a **cache memory hierarchy** (Section 11).

10. Principle of Locality

Principle of Locality: Programs tend to **reuse data and instructions** that are **close** (in address or time) to those that have been recently referenced.

This principle is the **fundamental justification** for cache memory hierarchies. It exists in two forms:

Temporal Locality

Definition: A recently referenced item is likely to be referenced again **in the near future**.

- **Cause:** Programs use loops, recursive functions, and frequently accessed variables.
- **Example:** The variable `sum` is accessed on *every* iteration of a loop → it will be needed again very soon.

Spatial Locality

Definition: Items at **memory addresses nearby** to a recently referenced address are likely to be referenced soon.

- **Cause:** Programs access arrays sequentially, execute instructions sequentially (**program counter** increments), and use data structures stored in contiguous memory.
- **Example:** Accessing `a[0]`, then `a[1]`, `a[2]` ... in sequence.

Worked Example from Slides

```
int sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

Access Pattern	Type of Locality
<code>a[0]</code> , <code>a[1]</code> , <code>a[2]</code> ... accessed in order	Spatial (adjacent array elements)
<code>sum</code> is read and written on <i>every</i> loop iteration	Temporal (same variable, used repeatedly)
Instructions in the loop body are fetched sequentially	Spatial (adjacent instruction addresses)
The loop body executes repeatedly (same instructions re-fetched)	Temporal (same instructions, executed again and again)

How Locality Bridges the CPU–DRAM Gap

- Because most accesses exhibit locality, a **small, fast cache** can be loaded with the *likely-needed* data.
- Most memory accesses → **cache hit** (served in 1–4 cycles) rather than going to slow **DRAM** (100+ cycles).
- This makes the **effective average memory access time** close to cache speed, not **DRAM** speed.

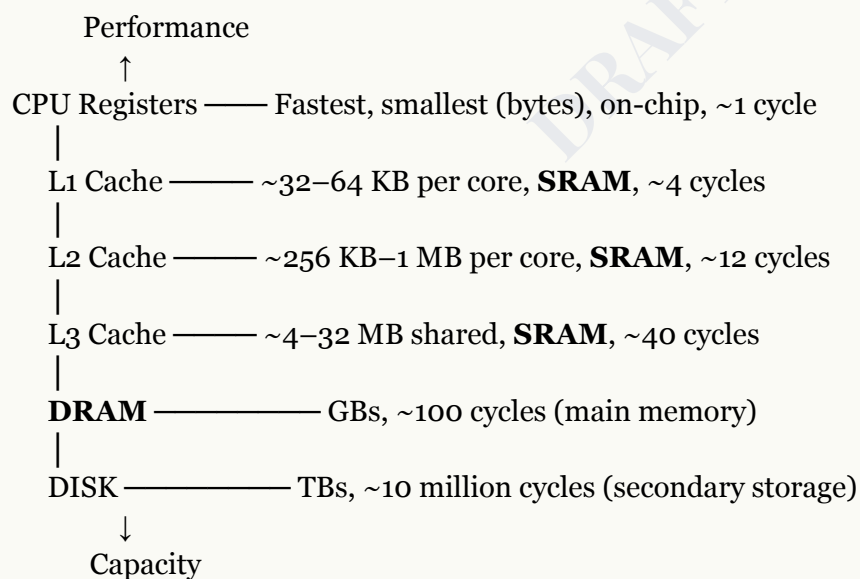
11. Memory / Storage Hierarchies

Core Concept

Property	Explanation
Smaller memories are faster but expensive	SRAM (cache): nanosecond access, ~\$10/MB
Larger memories are slower but cheaper	DRAM (main memory): ~100ns access, ~\$0.01/MB
Disk storage is very large, very slow, very cheap	HDD: millisecond access, ~\$0.00003/MB

Strategy: Exploit locality — keep the most frequently accessed data in the smallest, fastest memory level.

Memory Hierarchy (from registers to disk):



Course Note: The RP2040 **microcontroller** used in this course is itself a **dual-core** processor — an example of a multi-core **embedded system**.

11.1 Three Properties of Memory Levels

Every level in the **memory hierarchy** must satisfy three fundamental properties:

Property 1: Inclusion

Any data that is present at a **lower level** (closer to the CPU, i.e., L1 cache) **must also be present at all higher levels** (L2, L3, **DRAM**).

- If a cache block is in L1, the same block must also exist in L2, L3, and **DRAM**.
- This ensures that data evicted from L1 can always be found in L2.

Property 2: Coherence (Consistency)

Multiple copies of the same data may exist simultaneously at different levels of the hierarchy. **All copies must be kept identical (consistent)** during program execution.

- Writes to a cache block must eventually propagate to all other copies.
- In multi-processor systems, multiple CPUs may each have a copy of the same memory block in their private L1 caches — **cache coherence protocols** (e.g., MESI protocol) ensure consistency.
- This issue is called "**cache coherency**" — a critical problem in multiprocessor system design.

Property 3: Locality of Reference

Three sub-types:

Sub-Type	Definition
Temporal	Recently used data/instructions are likely to be reused soon
Spatial	Data at addresses adjacent to recently accessed addresses are likely to be accessed soon
Sequential	Instructions are typically executed in sequential order (PC increments linearly)

11.2 Quantitative Relationships Between Memory Levels

For adjacent memory levels i (closer to CPU) and $i+1$ (farther from CPU):

Property	Symbol	Relationship	Interpretation
----------	--------	--------------	----------------

Access time	t	$t_i < t_{i+1}$	Level closer to CPU is faster
Cost per byte	c	$c_i > c_{i+1}$	Level closer to CPU is more expensive
Memory size	s	$s_i < s_{i+1}$	Level closer to CPU is smaller
Transfer bandwidth	b	$b_i > b_{i+1}$	Closer level has higher bandwidth
Unit of transfer	x	$x_i < x_{i+1}$	Closer level transfers smaller chunks (e.g., bytes vs. cache lines vs. pages)
Frequency of access	f	$f_i > f_{i+1}$	Closer level is accessed more often (due to locality)

Access time: The time elapsed from when a memory request is issued to when the data is available.

Bandwidth unit: Bytes/Second.

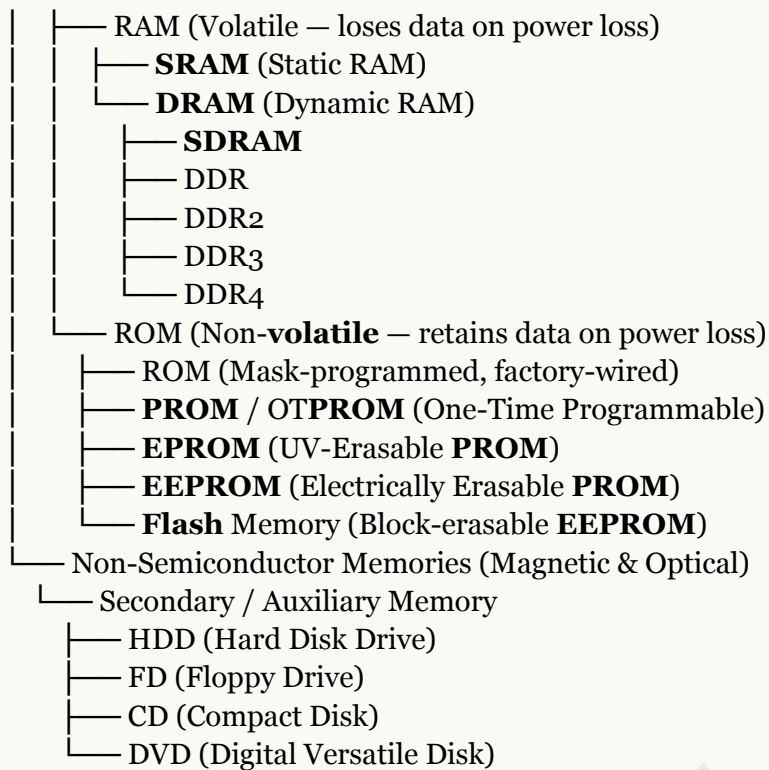
11.3 Cache Coherency Problem

- Cache sizes are always **powers of two**: e.g., L1 = 32 KB, L2 = 256 KB, L3 = 2 MB
- Higher-level cache sizes are **multiples of lower-level cache sizes**
- When data is brought into L1 cache, the **same copy exists in L2, L3, and DRAM**
- Having multiple copies of the same data raises the problem:
 - If CPU core 0 modifies a cache block in its L1, the copies in L2/L3 and in other cores' caches are now **stale (incorrect)**
 - This **inconsistency** must be detected and resolved by **cache coherency hardware** (e.g., snooping bus protocols, directory protocols)
- **Cache coherency** is a major design challenge in multiprocessor/multicore system design

12. Memory Classification

12.1 Full Taxonomy Tree

Memory
 |— Semiconductor Memories



Key abbreviations:

Abbreviation	Full Form
EPROM	Erasable Programmable ROM
UVEPROM	Ultra-Violet Erasable PROM
EEPROM	Electrically Erasable PROM
OTPROM / OTP NVM	One-Time Programmable Non-Volatile Memory
Flash	Flash Memory (NAND or NOR flash)

12.2 Volatile vs. Non-Volatile Memory

Property	Volatile Memory	Non-Volatile Memory
----------	-----------------	---------------------

Data retention on power loss	Lost immediately when power is cut	Retained even when power is off
Primary use	Main memory (working memory for running programs)	Program storage, secondary storage, persistent data
Examples	SRAM, DRAM	Flash, EEPROM, EPROM, ROM, PROM, HDD, SSD
CPU access speed	Fast (directly accessible by CPU)	Slower (Flash: μs – ms ; HDD: ms)
Exception	—	Flash drives are non-volatile <i>and</i> electrically erasable (unlike true ROM)

Laptop analogy (from slides):

- **Sleep mode:** Power to display and peripherals is cut, but **DRAM retains power** → contents of running programs are preserved → resumes quickly.
- **Hibernate mode:** Contents of **DRAM** are first **copied to HDD**, then **DRAM** is powered off → takes longer to resume because OS must copy program contents back from HDD to RAM before execution can continue.

12.3 Semiconductor vs. Non-Semiconductor Memories

Property	Semiconductor Memories	Non-Semiconductor Memories
Technology	Semiconductor (transistor-based)	Magnetic (HDD, tape) or Optical (CD, DVD)
Access type	Random access (any address in same time)	Normally sequential access (seek time varies by position)
Volatility	Mostly volatile (Flash is exception: non-volatile)	Non-volatile
Access time	Much lower (ns range) → less time to read	Higher (ms range) → more time to read
Proximity to CPU	Closer (SRAM is on the same chip as CPU)	Farther (built as a separate external unit)
Capacity	GBs (10^9 bytes)	TBs (10^{12} bytes) and beyond

13. Read-Only Memory (ROM) Types

ROM is a Programmable Logic Device (PLD): It stores binary data which is either hard-wired at manufacture or programmed electrically after manufacture. ROM is **non-volatile**.

13.1 ROM (Mask-Programmed)

The data is **physically wired** into the chip during the **IC fabrication process** (mask design step).

- The "programming" is done by the IC manufacturer using photolithographic masks that create (or omit) transistor connections to encode 0s and 1s.
 - Once manufactured, **contents are permanent** — cannot be changed by the user or by software.
 - **Write capability:** None — strictly read-only after fabrication.
 - **Applications:**
 - Storing **microprograms / microcode** inside **microprocessors** (sequence of μ -ops that implement assembly instructions in a CPU)
 - BIOS **firmware** in early PC systems
 - Fixed lookup tables (e.g., trigonometric tables in calculators)
 - **Economics:** Cheap per unit at **very high production volumes** (NRE costs amortized); expensive for small runs.
-

13.2 PROM (Programmable ROM / OTP)

Like ROM, **PROM** is **non-volatile** and can be **written only once** — but the writing is done **electrically** (not during manufacture).

- Ships from factory as a blank chip (all storage cells in the same state — usually all 1s).
- Writing process: applying high-voltage pulses to **blow fusible links** (or anti-fuse them) to encode 0s into selected bit positions.
- Writing (programming) can be performed by the **supplier or the end user**, after chip fabrication.
- Requires a **special PROM programmer device** connected to a PC.
- Also called: **One-Time Programmable (OTP)** or **OTP NVM (non-volatile memory)**
- **Advantages over ROM:** More flexible (no NRE cost to reprogram), convenient for small production runs.
- **Disadvantages:** More expensive per chip than ROM; cannot be corrected if programmed incorrectly.
- **ROM remains preferred** for very high-volume production (lower per-unit cost).

13.3 EPROM (UV-Erasable PROM)

EPROM is optically (UV light) erasable and electrically programmable.

Key properties:

- Data is read and written **electrically** (like **PROM**) — used as normal memory chips on a circuit board.
- **Erase mechanism:** Expose the chip to **ultraviolet (UV) light** through a transparent quartz window on the chip package for approximately **20 minutes**.
- UV photons provide energy to discharge floating-gate transistors → all bits reset to **1** (erased state).
- **Constraint: Entire chip must be erased before any new data can be written** — selective byte erasure is not possible.
- After UV erasure → use a **PROM programmer** to write specific data.
- **EPROM can be erased and reprogrammed multiple times** (unlike **PROM**, which is OTP).
- **Cannot be erased while installed on the circuit board** — must be physically removed.
- **PROMs** are covered with **stickers** to prevent accidental erasure from ambient light.

EPROM Programming Sequence (from slides):

1. **ERASE the entire chip** — place in UV eraser for ~20 minutes; all bits → 1
2. **PROGRAM it** — use **PROM** programmer (connected to PC) to write specific 0s into target locations
 - *"Programming **EPROM** actually involves only writing zeros into specific locations"* — since erasing sets all bits to 1, programming is selectively writing 0s.
3. **USE it** — install back on board; operates as read-only memory during normal circuit operation

Cost: More expensive than **PROM**; justifies expense through multiple update capability.

13.4 EEPROM (Electrically Erasable PROM)

EEPROM is a **read-mostly non-volatile** memory that can be **written at any time without erasing the entire prior contents** — selective byte-level update is possible.

Key properties:

- Both read and write operations performed **electrically** (no UV required).
- Can update **individual bytes or a set of bytes** without disturbing the rest of the chip contents.
- **Write speed:** Considerably slower than read operations — write latency in the order of **several hundred microseconds (100s of μ s)**.
- **Advantage over EPROM:**
 - No need to remove chip from circuit — can be updated **in-circuit** (in-system programming, ISP).
 - Uses **ordinary bus control, address, and data lines** for in-circuit updates.
 - Selective byte update possible.
- **EEPROM combines:** Non-volatility + in-circuit electrical reprogrammability.
- **Disadvantages vs. EPROM:**
 - More **expensive** than **EPROM**
 - **Less dense** — fewer bits per chip (more complex cell structure)
- **Typical application:** Storing configuration parameters, calibration data, serial numbers, or small data tables in **embedded systems**.
- **Example chip from slides:** 24C64 **EEPROM** — 8-pin package, 2-wire serial interface (I²C), $64K \times 8$ bits = 64 Kbytes capacity.

13.5 Flash Memory

Flash memory is a type of **non-volatile** semiconductor memory derived from **EEPROM** technology, but optimized for **block-level erasure and high-density storage**.

Key properties:

- **Erased in blocks** (sectors) — not byte-by-byte like **EEPROM**, not whole-chip like **EPROM**.
- Read and written electrically; no UV required.
- **Non-volatile** — retains data without power.
- Used as the **secondary storage alternative to HDD** in **embedded systems** (no moving parts → better reliability, lower power, shock-resistant).
- **Examples:** USB memory sticks, SD cards, SSDs, **microcontroller** program memory (RP2040 uses external QSPI **Flash**).
- The RP2040 in the course uses **Flash** to store program code.

13.6 Comparison Table: All ROM Types

Property	ROM	PROM (OTP)	EPROM	EEPROM	Flash
Programmed by	Manufacturer (masks)	User (electrically, once)	User (electrically)	User (electrically)	User (electrically)
Erase method	Cannot erase	Cannot erase	UV light (~20 min)	Electrically (byte)	Electrically (block)
Erase granularity	—	—	Whole chip	Byte / few bytes	Block (sector)
Number of write cycles	0 (read-only)	1 (one-time)	Many (after each UV erase)	Many (~100K–1 M cycles)	Many (~10K–100K cycles)
In-circuit reprogramming	No	No	No (must remove chip)	Yes	Yes
Volatility	Non-volatile	Non-volatile	Non-volatile	Non-volatile	Non-volatile
Write speed	—	Slow (high-voltage fusing)	Slow (high-voltage fusing)	Slower than read (~100s μ s/byte)	Fast (block write)
Cost	Cheap (high volume)	Moderate	More expensive than PROM	More expensive than EPROM	Cheaper than EEPROM at scale
Density	High	High	High	Lower than EPROM	Very High
Primary use	Microcode, fixed firmware	One-time factory config	Development/prototyping firmware	Config data, calibration	Program storage, USB drives, SSDs

Yes/No Quick Reference (from Quiz slides):

Statement	Answer
ROM: Data is wired in during manufacture	YES
ROM: Can be written by the user	NO
ROM: Can store μ -ops in CPUs	YES
PROM: Writing = programming the chip	YES
PROM: Programming done during manufacture	NO
PROM-OTP: Cannot be written more than once	YES
PROM: More expensive than ROM	YES
EPROM: UV used to erase	YES
EPROM: Possible to erase part of the chip	NO (whole chip only)
EPROM: Reading/writing both done electrically	YES
EPROM: Can be written only after erasing all data	YES
EPROM: Can be erased in-circuit	NO (must remove from board)
EPROM: Programming writes only zeros	YES (erasing sets all to 1; programming drives selected bits to 0)
EEPROM: Erasing done optically	NO (electrically)
EEPROM: Erasing easier than EPROM	YES
EEPROM: Writing faster than reading	NO
EEPROM: More expensive than EPROM	YES
Writing is slower than reading on all ROM types	YES

14. Random Access Memory (RAM)

Definition: RAM (Random Access Memory) is a semiconductor memory where the time taken to **read or write** a data item at **any location** is the **same**, regardless of the physical location of that item in the memory array.

Key property: Access time is **location-independent** — uniform access time across all addresses.

Contrast with sequential-access storage media:

- **HDD, CD, DVD, magnetic tape:** Access time varies significantly depending on the physical location of the data on the recording medium (seek time + rotational latency for HDD).
- **RAM:** No mechanical movement → constant access time.

14.1 SRAM (Static RAM)

SRAM stores each bit in a **bistable flip-flop circuit** (typically 6 transistors per cell — **6T SRAM**). Data is retained as long as power is applied, without the need for periodic refresh.

Key characteristics:

Property	Value / Description
Cell structure	6-transistor (6T) bistable latch
Data retention	Static — holds data as long as VCC is present; no refresh required
Volatility	Volatile (data lost on power off)
Access time	Very fast — ~1 to 10 ns (same order as CPU pipeline stages)
Power consumption	Low (static power only; no switching for refresh)
Area per bit	Large (6 transistors) → low density
Cost per bit	Expensive (large area → fewer bits per chip)
Typical sizes	KBs to low MBs
Primary use	CPU register files, L1/L2/L3 cache memories
Location in system	On-chip (integrated into the same die as the processor)

14.2 DRAM (Dynamic RAM)

DRAM stores each bit as a **charge on a capacitor**, with one transistor per cell (1T1C). The charge leaks away over time, requiring periodic **refresh cycles** to restore the data.

Key characteristics:

Property	Value / Description
Cell structure	1 transistor + 1 capacitor (1T1C)
Data retention	Dynamic — capacitor charge leaks; requires refresh every ~64 ms
Volatility	Volatile (data lost on power off AND if refresh fails)
Access time	Slower — ~50 to 100 ns
Power consumption	Higher (refresh operations consume power continuously)
Area per bit	Small (1T1C) → high density
Cost per bit	Cheaper (small area → many bits per chip)
Typical sizes	GBs (main memory in computers)
Primary use	Main memory (DRAM DIMMs in PCs; LPDDR in mobile/embedded)
Location in system	Off-chip (separate memory chip/DIMM connected via memory bus)

Why refresh is needed:

- The capacitor in each **DRAM** cell slowly **discharges** (leaks charge) due to parasitic currents.
 - If the charge drops below the sensing threshold, the bit value is lost.
 - The **DRAM** controller must **periodically read and rewrite** each row to refresh the charge — this is called a **DRAM refresh cycle**.
 - Refresh operations temporarily **block normal memory access**, contributing to **DRAM** latency.
-

14.3 SRAM vs. DRAM Comparison Table

Property	SRAM	DRAM
Storage element	6T bistable latch (flip-flop)	1T1C (transistor + capacitor)
Refresh required	No	Yes (~64 ms period)
Access time	Fast (~1–10 ns)	Slow (~50–100 ns)
Cost per bit	High (expensive)	Low (cheap)
Density	Low (large cell area)	High (small cell area)
Power consumption	Low (no refresh overhead)	Higher (continuous refresh overhead)
Typical capacity	KBs – low MBs	GBs
Primary application	CPU caches (L1, L2, L3)	Main memory
Location in system	On-chip (same die as CPU)	Off-chip (separate DIMM/chip)
Volatility	Volatile	Volatile
Complexity	Higher (complex cell)	Lower (simple cell)

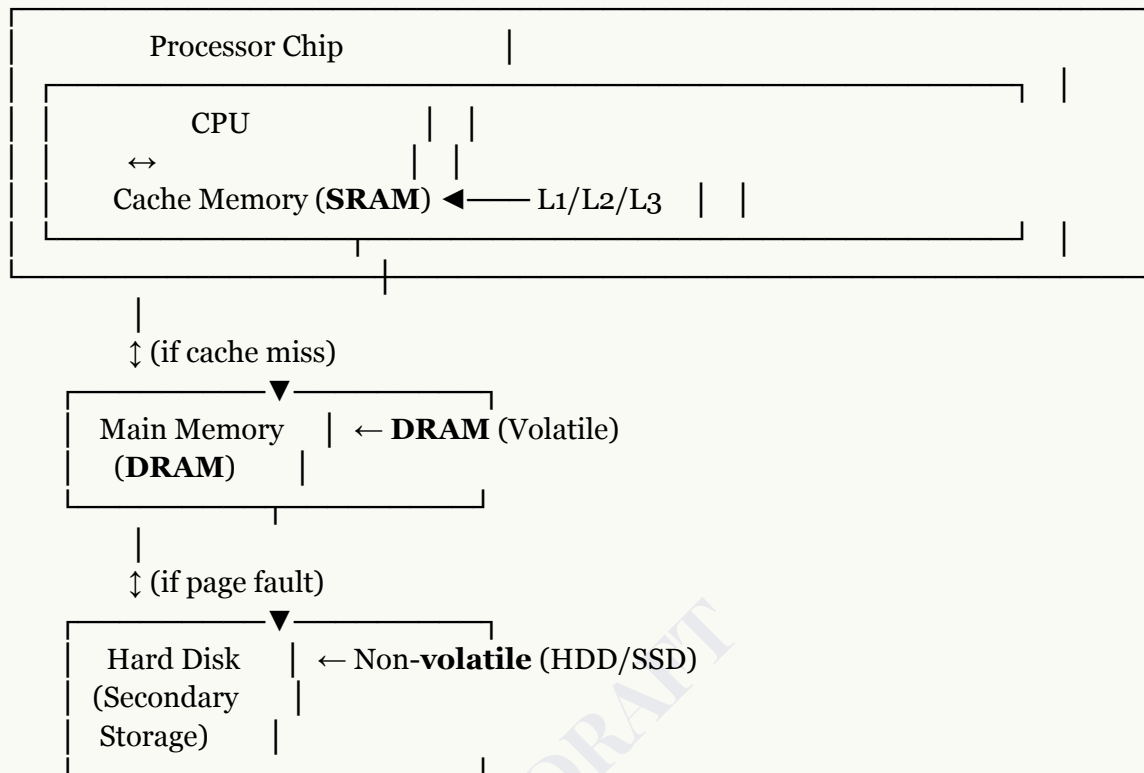
14.4 DRAM Variants

The **DRAM** family has evolved through several generations to increase bandwidth:

Variant	Full Name	Key Improvement
SDRAM	Synchronous DRAM	Synchronized with CPU clock bus; eliminates asynchronous wait states
DDR	Double Data Rate SDRAM	Transfers data on both rising and falling edges of the clock → 2× bandwidth of SDRAM
DDR2	DDR2 SDRAM	Higher clock speeds, lower operating voltage (1.8V vs. DDR's 2.5V), prefetch buffer: 4n
DDR3	DDR3 SDRAM	Even higher speeds, 1.5V operation, prefetch buffer: 8n
DDR4	DDR4 SDRAM	Higher bandwidth (up to 3200 MT/s), 1.2V operation, improved power efficiency

14.5 Use of SRAM and DRAM in a System

The **three-level search hierarchy** for data:



Search sequence:

1. CPU first seeks data in the **on-chip cache (SRAM)** → typically found here (due to locality)
 2. If **cache miss** → CPU looks in **main memory (DRAM)**
 3. If **page fault** (not in **DRAM** either) → OS fetches the data from **hard disk** into **DRAM** (extremely slow)
-

15. Quick Revision Cheatsheet

Embedded System Key Facts

Fact	Detail
Definition	Microcontroller/microprocessor-based, specific-function, usually battery-operated system
Three components	Hardware + Embedded SW + RTOS
Bare metal	SW runs directly on HW with no OS
RTOS purpose	Scheduling, latency enforcement, inter-task communication
Design metrics	Cost, size, power, latency, performance

Characteristics Summary

Characteristic	One-line Description
Single-functioned	Does one job repeatedly (e.g., pager = pager, always)
Tightly constrained	Missing deadlines = disaster; tight cost/size/power budgets
Reactive & real-time	Continuously reacts to environment; results must be computed within deadlines
Microprocessor-based	Must have an MCU/MPU running RTOS or bare-metal firmware
Memory-specific	Flash/ROM for program; SRAM/DRAM for working memory; no HDD
Connected	Interfaces to peripherals (I ² C, SPI, UART, etc.); IoT → network
HW-SW system	HW for performance/security; SW for flexibility/features

Memory Hierarchy Quick Facts

Level	Type	Speed	Size	Use
Registers	SRAM (on-chip)	~1 cycle	Bytes	Immediate operands
L1 Cache	SRAM (on-chip)	~4 cycles	32–64 KB	Hot data/code

L2 Cache	SRAM	~12 cycles	256 KB–1 MB	Warm data
L3 Cache	SRAM	~40 cycles	4–32 MB	Shared by all cores
Main Memory	DRAM (off-chip)	~100 cycles	GBs	All running programs
Disk	Non-semiconductor	~10M cycles	TBs	Files, OS, swap

ROM Types One-Line Summary

Type	Erase Method	Erase Granularity	Write Cycles	In-Circuit ?
ROM	None	—	0	No
PROM	None	—	1	No
EPROM	UV (~20 min)	Whole chip	Many	No
EEPROM	Electrical	Byte	~100K–1M	Yes
Flash	Electrical	Block (sector)	~10K–100K	Yes

Locality Types

Type	Definition	Example
Temporal	Recently accessed items reused soon	Variable <code>sum</code> in loop
Spatial	Nearby addresses accessed together	Array <code>a[i]</code> in sequence
Sequential	Instructions fetched in order	Linear program execution

Memory Level Relationships (i = closer to CPU, $i+1$ = farther)

Access time: $t_i < t_{i+1}$ (closer = faster)
 Cost per byte: $c_i > c_{i+1}$ (closer = costlier)
 Memory size: $s_i < s_{i+1}$ (closer = smaller)
 Bandwidth: $b_i > b_{i+1}$ (closer = higher BW)
 Unit of transfer: $x_i < x_{i+1}$ (closer = finer granularity)
 Frequency: $f_i > f_{i+1}$ (closer = accessed more often)

C Program Memory Layout

High address → Stack (local vars, grows ↓)

Heap (malloc, grows ↑)

BSS (uninit globals/statics → zero-filled by OS)

Initialized Data (init globals/statics → from .exe)

Low address → Code/Text (program opcodes → from .exe)

CPU-DRAM Gap

CPU improves ~60%/year | **DRAM** improves <10%/year → ever-growing gap

Solution: Cache hierarchy exploiting principle of locality

*End of ESM Unit 1 Master Notes — CS1120, RV University Prepared for CIE 1 | Coverage:
Introduction to Embedded Systems & Memory*

DRAFT